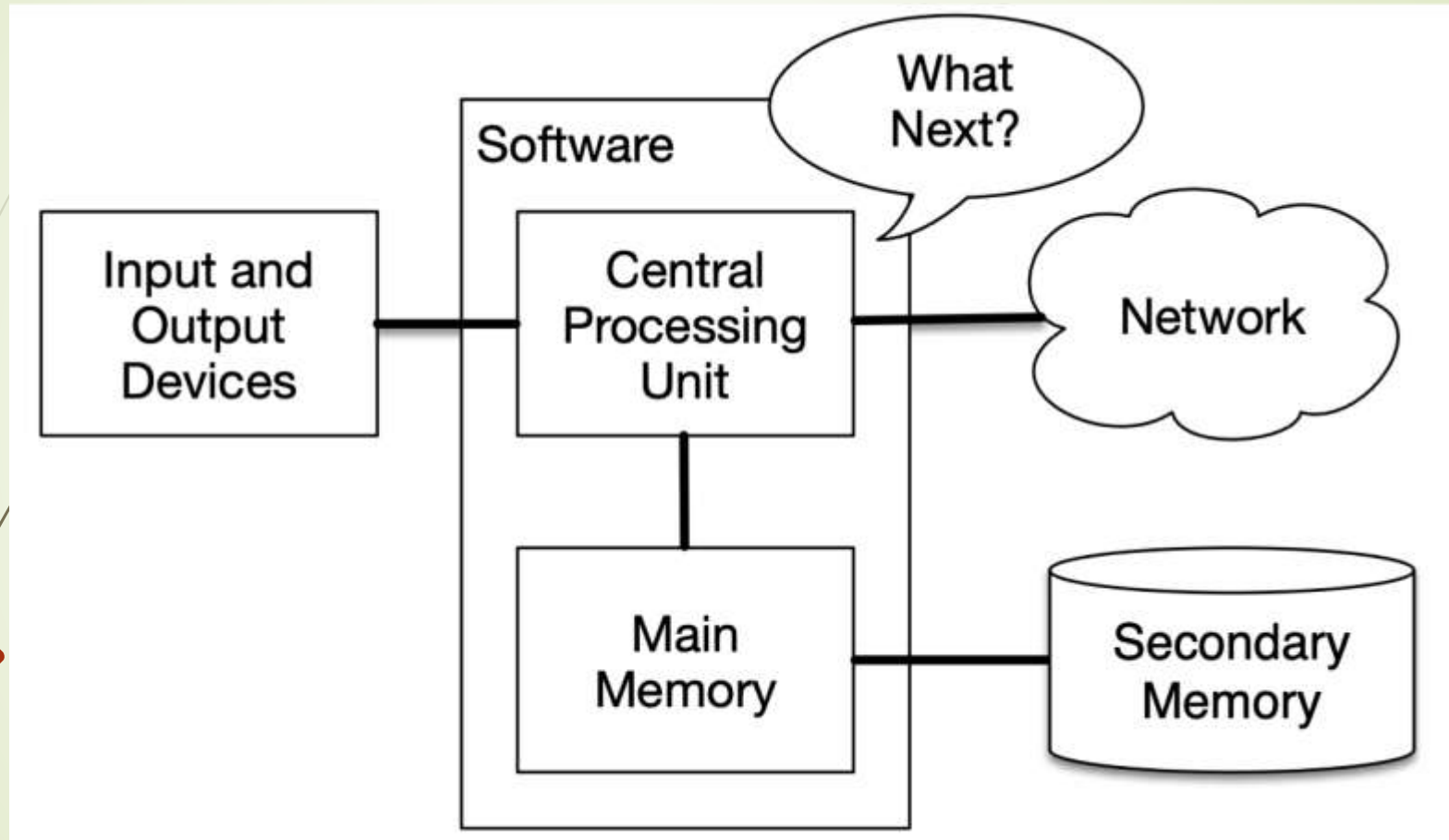

Nhắc lại kiến thức bài trước

File I/O & Kiểu dữ liệu



Bộ nhớ thứ cấp (Secondary Memory)



Bộ nhớ thứ cấp (Secondary Memory)

write() method

- Câu sau mở tệp tin **python.txt** ở chế độ ghi (write mode).

```
>>> f=open("python.txt","w")
```

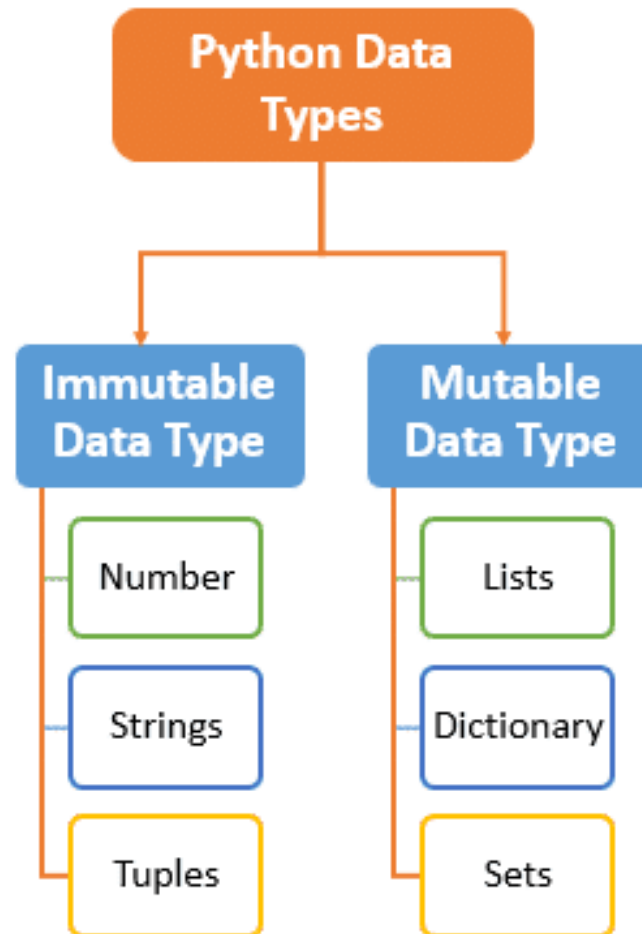
- Tiếp theo chúng ta cần đưa dữ liệu vào tệp tin. Phương thức **write()** lưu trữ một chuỗi (string) vào tệp tin.

```
>>> f.write("Monty Python's Flying Circus")
```

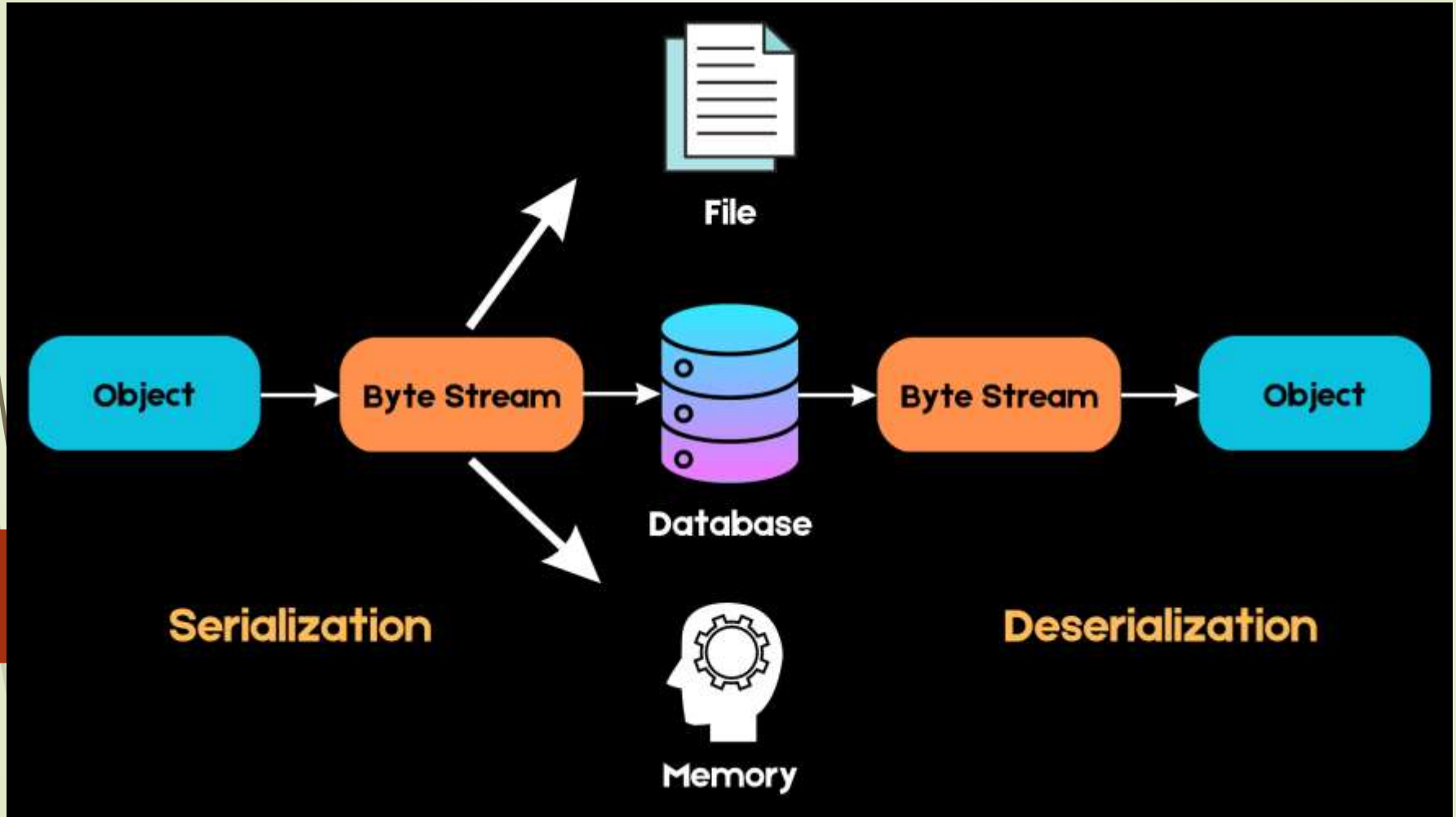
- Hãy đảm bảo rằng bạn đóng đối tượng tệp tin vào cuối cùng.

```
>>> f.close()
```

Sự khác nhau giữa List và Tuples



Object Serialization - Deserialization



Sinh viên tìm hiểu cách tuần tự hóa/giải tuần tự các đối tượng với JSON file

JSON — Lưu/Tải Danh Sách Cán Bộ

Chuyển đối tượng → dict, rồi dump ra file

python

```
import json

class CanBo:
    def __init__(self, ho_ten, tuoi,
                  gioi_tinh, dia_chi):
        self.ho_ten = ho_ten
        self.tuoi = tuoi
        self.gioi_tinh = gioi_tinh
        self.dia_chi = dia_chi

    def to_dict(self):          # Bước quan trọng!
        return {
            "ho_ten": self.ho_ten,
            "tuoi": self.tuoi,
            "gioi_tinh": self.gioi_tinh,
            "dia_chi": self.dia_chi,
            "loai": self.__class__.__name__,
        }

    @classmethod
    def from_dict(cls, d):     # Phục hồi từ dict
        return cls(d["ho_ten"], d["tuoi"],
                   d["gioi_tinh"], d["dia_chi"])
```

python

```
# === LƯU ===
danh_sach = [
    CanBo("Nguyễn An", 30, "Nam", "Hà Nội"),
    CongNhan("Trần Bình", 25, "Nam", "TP.HCM", 5),
]

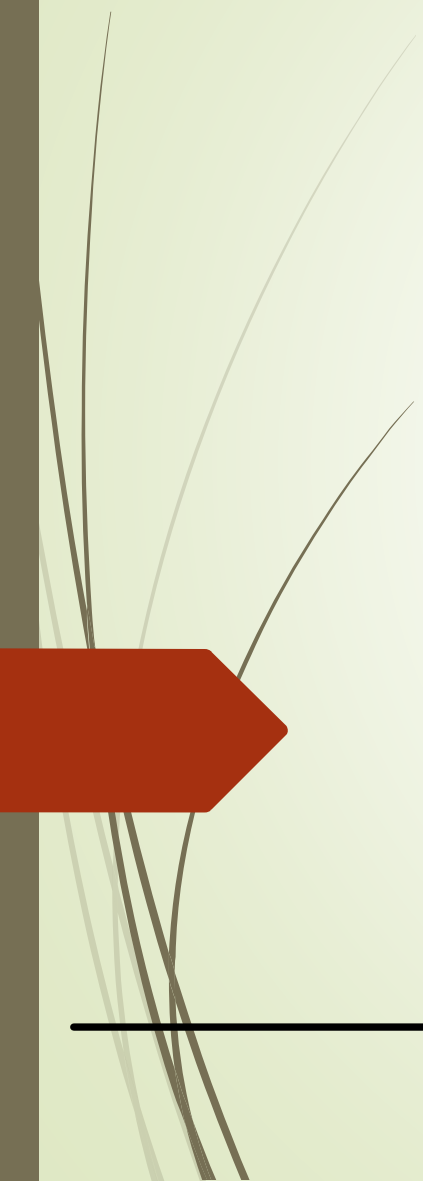
data = [cb.to_dict() for cb in danh_sach]

with open("canbo.json", "w",
          encoding="utf-8") as f:
    json.dump(data, f,
              ensure_ascii=False, indent=2)

# === TẢI ===
with open("canbo.json", "r",
          encoding="utf-8") as f:
    raw = json.load(f)

# Khôi phục đúng loại theo "loai"
ds_loaded = [CanBo.from_dict(d)
              for d in raw]

for cb in ds_loaded:
    print(cb)
```



OOP08: UML

Bài giảng e-learning đi kèm (1)

Hiện tại bài giảng “**Software Architecture & Design**” của Udemy đã được full upload và free trên youtube. Sinh viên tham khảo phần **UML** và các biểu đồ liên quan. Khuyến khích xem tất cả các nội dung của bài giảng.

https://www.youtube.com/watch?v=ct7ZlwdtXe8&list=P_LAwxTw4SYaPkMTetlG7xKWaI5ZAZFX8fL&index=49



Bài giảng e-learning đi kèm (2)

- Bài giảng của Smartdraw
 - <https://www.smartdraw.com/uml-diagram/>
 - <https://www.smartdraw.com/use-case-diagram/>
 - <https://www.smartdraw.com/activity-diagram/>
 - <https://www.smartdraw.com/sequence-diagram/>
 - <https://www.smartdraw.com/class-diagram/>



Nội dung

1. UML là gì?
 2. Các biểu đồ UML cơ bản
 3. Giới thiệu biểu đồ Use case
 4. Giới thiệu biểu đồ Lớp
 5. Giới thiệu biểu đồ Tương tác
 6. Giới thiệu biểu đồ Hoạt động
- 

1

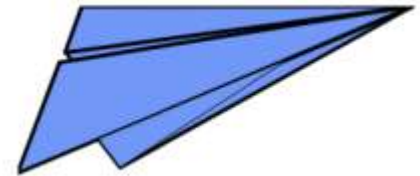
UML là gì?

Unified Modeling Language

1.1. Tầm quan trọng của phân tích và thiết kế

- **Hướng tiếp cận không có phân tích – thiết kế:**

- Bắt đầu lập trình ngay khi có được yêu cầu
- Mất rất nhiều thời gian và tạo đi tạo lại nhiều mã nguồn
- Không có bất kỳ một kiến trúc nào
- Phải chịu khổ với những lỗi phát sinh



- **Hướng tiếp cận có phân tích – thiết kế:**

- Chuyển các yêu cầu của bài toán thành một bản thiết kế rõ ràng
- Tập trung vào phân tích các YÊU CẦU và thiết kế các MÔ HÌNH cho hệ thống TRƯỚC khi lập trình

1.1. Tầm quan trọng của phân tích và thiết kế

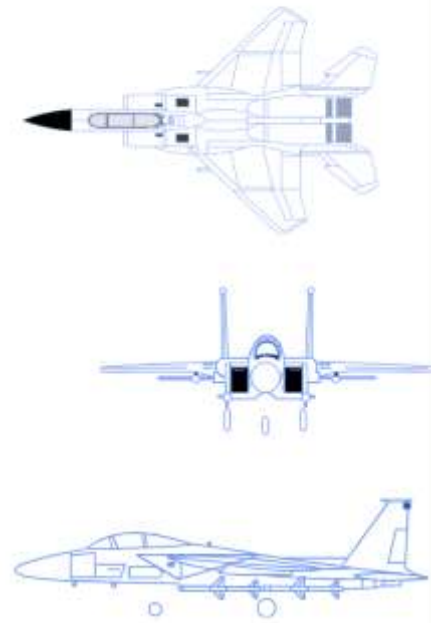
- **Ưu điểm của việc PTTK hệ thống:**

- Đơn giản hóa thế giới thực bằng các mô hình
- Mô tả đúng, đồng nhất cấu trúc, cách ứng xử của HT trong suốt quá trình xây dựng
- Đảm bảo mục đích và yêu cầu của HT được thỏa mãn trước khi xây dựng
- Cung cấp cho người dùng, khách hàng, kỹ sư phân tích, thiết kế, kỹ sư lập trình nhiều cái nhìn khác nhau về cùng một HT
- Ghi lại các quyết định của nhà phát triển để sử dụng sau này

Máy bay phản lực



Các góc nhìn



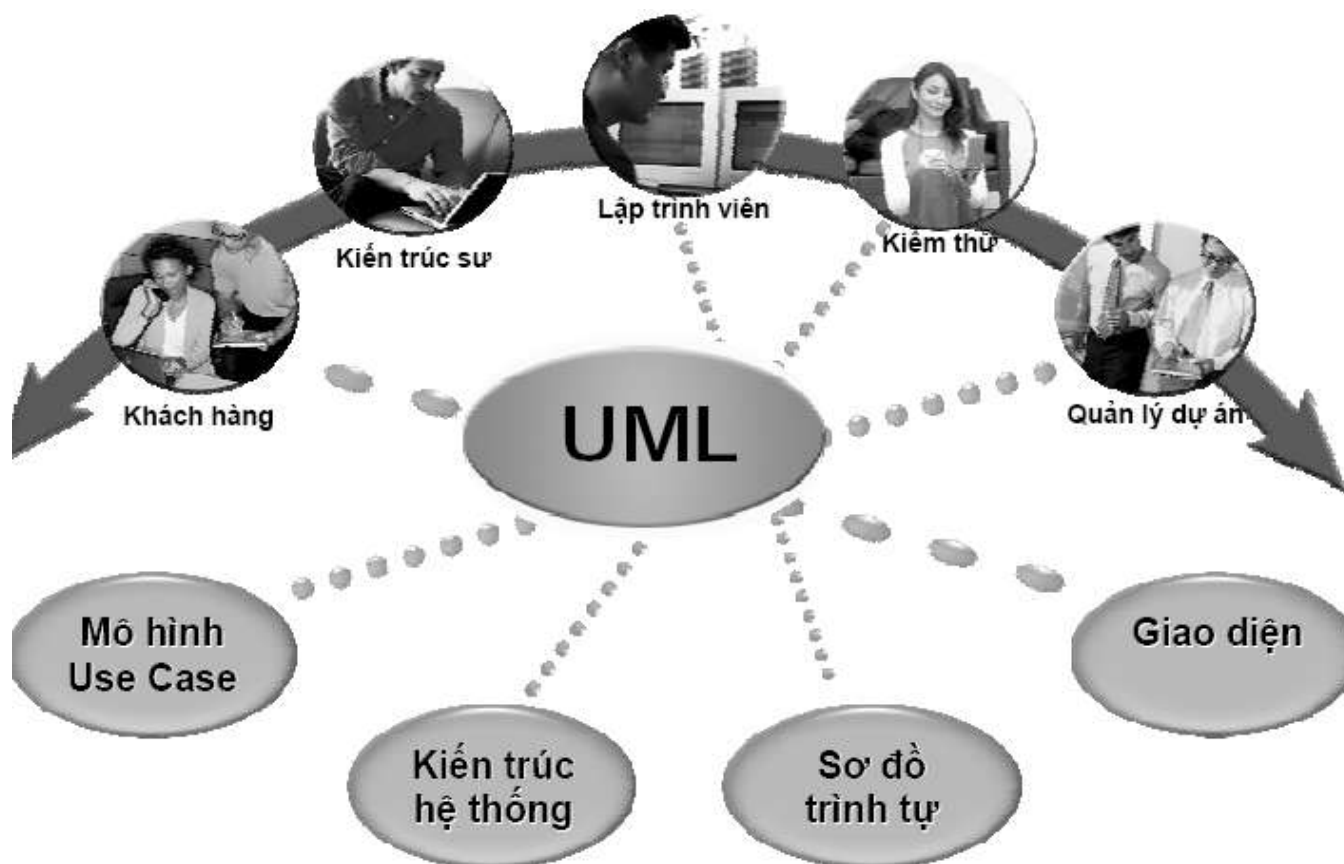
1.2. Khái niệm UML

- UML: viết tắt của “Unified Modeling Language” là một Ngôn ngữ mô hình hóa được thống nhất
- UML là ngôn ngữ trực quan để:
 - trực quan hóa (visualizing)
 - đặc tả (specifying)
 - xây dựng (constructing)
 - tài liệu hóa (documenting)các cấu phần của một hệ thống phần mềm
- Giúp công việc phát triển được xử lý nhất quán, giảm thiểu lỗi xảy ra
 - Giúp dễ hình dung hơn cấu trúc của hệ thống
 - Hiệu quả hơn trong việc liên lạc, trao đổi



1.2. Khái niệm UML

- Thiết lập một phương thức thống nhất để xây dựng và “vẽ” ra các yêu cầu và thiết kế hướng đối tượng trong quá trình PTTK phần mềm

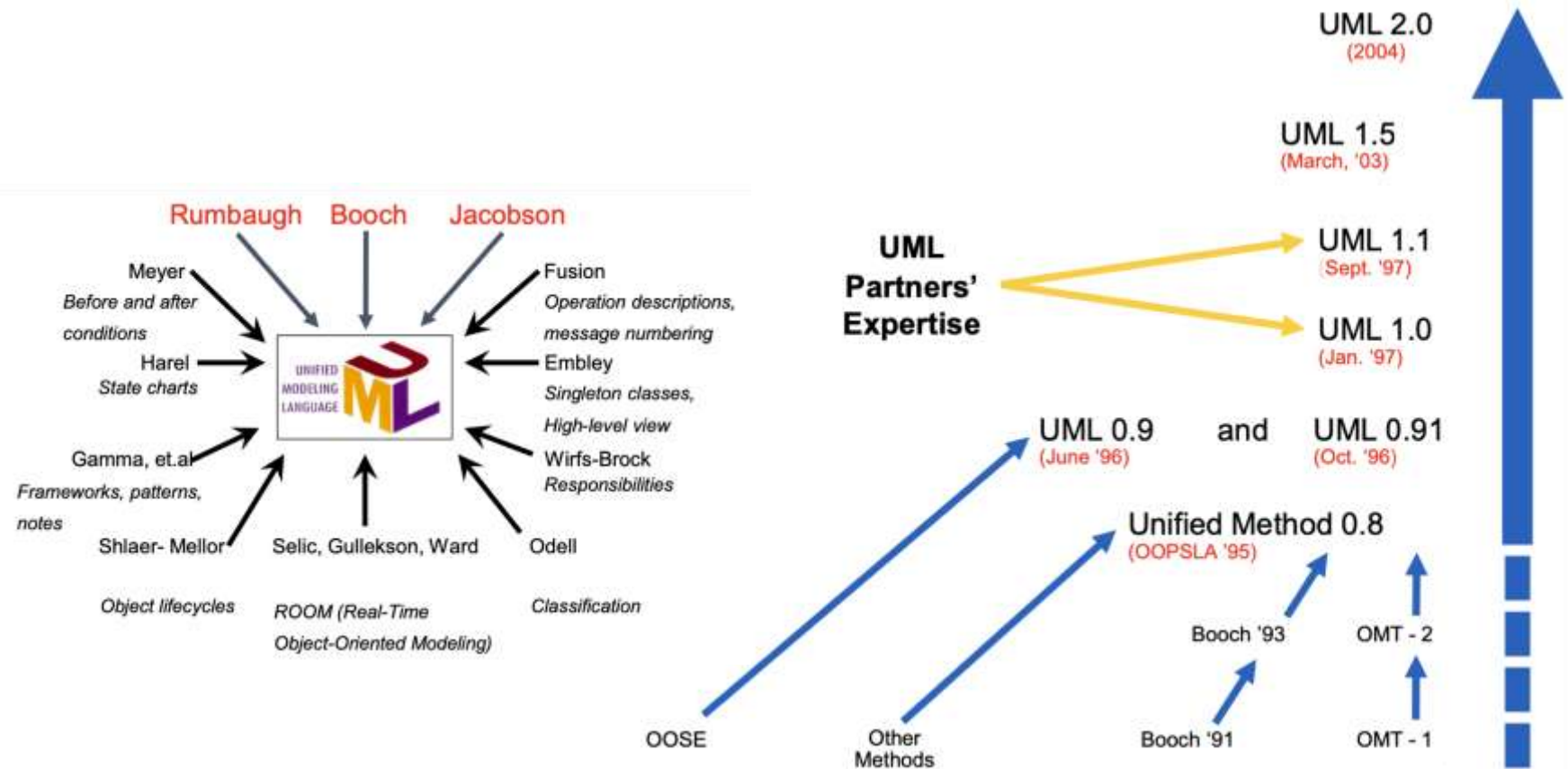


1.3. Lịch sử phát triển UML

- Vào năm 1994, có hơn 50 phương pháp mô hình hóa hướng đối tượng:
 - Fusion, Shlaer-Mellor, ROOM, Class-Relation, Wirfs-Brock, Coad-Yourdon, MOSES, Syntropy, BOOM, OOSD, OSA, BON, Catalysis, COMMA, HOOD, Ooram, DOORS ...
 - Mô tả về mô hình “Meta-models” tương đồng với nhau
 - Các ký pháp đồ họa khác nhau
 - Quy trình khác nhau hoặc không rõ ràng→ **Cần chuẩn hóa và thống nhất các phương pháp**
- UML được 3 chuyên gia hướng đối tượng hợp nhất các kỹ thuật của họ vào năm 1994:
 - Booch91 (Grady Booch): Conception, Architecture
 - OOSE (Ivar Jacobson): Use cases
 - OMT (Jim Rumbaugh): Analysis

1.3. Lịch sử phát triển UML

- UML là ngôn ngữ hợp nhất các mô hình khác nhau. Được công nhận là chuẩn chung vào năm 1997



1.4. Làm việc với UML

- Các mô hình UML có thể kết nối trực tiếp với rất nhiều ngôn ngữ lập trình.
 - Ánh xạ sang Java, C#, C++, Python, PHP, Hibernate, Ruby, VB.NET, .NET dll or exe, ODL, ActionScript, IDL, Delphi, Perl, XML, XML Schema, Objective-C 2.0, Ada95.
 - Các bảng trong RDBMS hoặc kho lưu trữ trong OODBMS
 - Cho phép các kỹ nghệ xuôi (chuyển UML thành mã nguồn)
 - Cho phép kỹ nghệ ngược (xây dựng mô hình hệ thống từ mã nguồn)

1.4. Làm việc với UML

NHÓM 1 — MÃ NGUỒN MỞ (OPEN SOURCE)

StarUML

Free trial

Desktop. Chuẩn UML nhất, xuất PNG/SVG đẹp. Khuyến nghị cho SV.

Astah UML

Free for SV

Desktop. Giao diện thân thiện, phổ biến trong giảng dạy.

ArgoUML

Free

Desktop Java. Hỗ trợ reverse engineering từ code.

PlantUML

Free

Text-to-diagram. Dùng tốt với AI — paste prompt, render ngay tại plantuml.com.

NHÓM 2 — THƯƠNG MẠI / FREEMIUM CHUYÊN NGHIỆP

Enterprise Architect

Trả phí

Công nghiệp. Đầy đủ nhất, dùng trong dự án lớn.

Visual Paradigm

Community free

Desktop + Online. Nay tích hợp AI gợi ý diagram.

Microsoft Visio

Trả phí

Phổ biến trong doanh nghiệp, tích hợp Office 365.

SmartDraw

Freemium

Online. Nguồn tài liệu học UML rất phong phú.

1.4. Làm việc với UML

NHÓM 3 — CỘNG TÁC & TƯ DUY TRỰC QUAN (MIRO, XMIND...)

Miro

Cộng tác AI

Online whiteboard. Vẽ UML từ text prompt, cộng tác real-time nhiều người. Tích hợp AI tạo diagram.

Lucidchart

Cộng tác Free SV

Online. Template đẹp, cộng tác real-time, tích hợp Google Drive.

XMind

Freemium

Mind map + sơ đồ. Phù hợp brainstorm, phân tích yêu cầu trước khi vẽ UML.

draw.io / diagrams.net

Free hoàn toàn

Online + Desktop. Không cần đăng ký. Khuyến nghị cho bài cá nhân.

Mermaid.js

Free

Text trong VS Code / GitHub Markdown. Diagram sống trong code repository.

NHÓM 4 — AI-POWERED UML (2024–2025)

Eraser / DiagramGPT

AI Free tier

Nhập tiếng Anh → AI tạo Use Case, Sequence, ERD, Class diagram ngay lập tức.

Claude / ChatGPT + PlantUML

AI Free

Prompt AI mô tả bài toán → nhận PlantUML code → paste vào plantuml.com để render.

Miro AI Diagramming

AI Cộng tác

Mô tả hệ thống → AI vẽ UML trực tiếp trên canvas, chỉnh sửa được ngay.

Visual Paradigm AI

AI Community free

AI gợi ý quan hệ lớp, validate diagram, xuất PlantUML/SVG. Tích hợp trong tool quen thuộc.

Diagramming AI

AI Free

Tạo và sửa Mermaid, PlantUML, Graphviz từ tiếng Anh. Hỗ trợ image-to-diagram.

DocuWriter.ai

AI Freemium

Tạo Class/Sequence diagram trực tiếp từ source code. Đồng bộ diagram khi code thay đổi.

2

Các biểu đồ UML cơ bản

2.1. Biểu đồ UML

- **Biểu đồ:**
 - là các hình vẽ bao gồm các ký hiệu phần tử mô hình hóa
 - minh họa một thành phần cụ thể hay một khía cạnh cụ thể của hệ thống.
- Một mô hình hệ thống thường có nhiều loại biểu đồ, mỗi loại gồm nhiều biểu đồ khác nhau.
- Một biểu đồ là một thành phần của một hướng nhìn cụ thể
- Một số loại biểu đồ có thể là thành phần của nhiều hướng nhìn khác nhau
- UML thế hệ 2 có tới 13-14 loại biểu đồ. Tuy nhiên, trong một project, chỉ sử dụng những biểu đồ phù hợp nhất

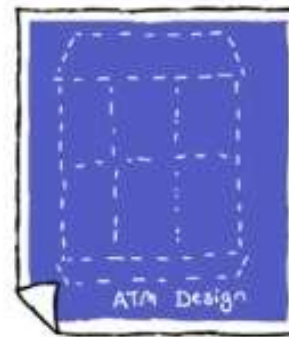
2.1. Biểu đồ UML

- **Phân biệt:**

- **Biểu đồ cấu trúc:** mô tả thành phần tĩnh, luôn có của hệ thống và mối quan hệ giữa chúng
- **Biểu đồ hành vi:** mô tả cách hoạt động của hệ thống.

Diagram Types

Two Main Categories:



Structural

what is contained in a system



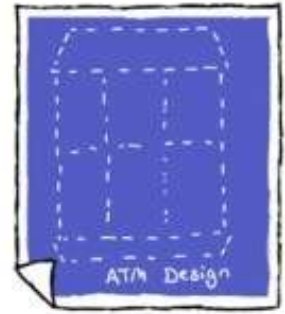
Behavioral

what must happen in a system

2.2. Các loại biểu đồ UML cơ bản

Biểu đồ cấu trúc - Tham khảo lesson 4

- Biểu đồ cấu trúc tĩnh
 - Biểu đồ lớp (Class Diagram) – (session) ss 6..9
 - Biểu đồ đối tượng (Object Diagram) – ss 10
 - Biểu đồ gói (Package diagram) - ss 16..17
- Biểu đồ thực thi
 - Biểu đồ thành phần (Component Diagram) – ss 12..13
 - Biểu đồ triển khai (Deployment Diagram) – ss 14..15
 - Biểu đồ cấu thành (Composite Diagram) – ss 11
- Biểu đồ profile (Profile Diagram) – ss 19



Structural

2.2. Các loại biểu đồ UML cơ bản

Biểu đồ hành vi - Tham khảo lesson 4

- Biểu đồ use case (Use Case Diagram) – ss 24..28
- Biểu đồ hoạt động (Activity Diagram) – ss 35..36
- Biểu đồ tương tác
 - Biểu đồ tổng quát (Interaction overview diagram) – ss 37
 - Biểu đồ trình tự (Sequence Diagram) – ss 31..32
 - Biểu đồ giao tiếp/cộng tác (Communication/Collaboration diagram) – ss 33..34
- Biểu đồ trạng thái (State Diagram) – ss 39..40
- Biểu đồ thời gian (Timing Diagram) – ss 38



Behavioral

2.2. Các loại biểu đồ UML cơ bản

Biểu đồ hành vi - Tham khảo lesson 4

- Biểu đồ use case (Use Case Diagram) – ss 24..28
- Biểu đồ hoạt động (Activity Diagram) – ss 35..36
- Biểu đồ tương tác
 - Biểu đồ tổng quát (Interaction overview diagram) – ss 37
 - Biểu đồ trình tự (Sequence Diagram) – ss 31..32
 - Biểu đồ giao tiếp/cộng tác (Communication/Collaboration diagram) – ss 33..34
- Biểu đồ trạng thái (State Diagram) – ss 39..40
- Biểu đồ thời gian (Timing Diagram) – ss 38



Behavioral

3

Biểu đồ Use case

Use case diagram

<https://www.smartdraw.com/use-case-diagram/>

<https://www.youtube.com/watch?v=Omp4RbHbB0s>

3.1. Khái niệm Use case

- Use case mô tả một chuỗi các hành động của người dùng cùng với chuỗi các đáp ứng/phản hồi của hệ thống để thực hiện 1 việc, 1 chức năng nào đó của hệ thống.
- Nội dung use case giống như kịch bản/đoạn hội thoại mô tả cách sử dụng hệ thống: luôn có cấu trúc “người dùng làm gì – hệ thống phản ứng như thế nào”.
- Cần mô tả kỹ về cách người dùng sử dụng hệ thống:
 - Người dùng làm theo kịch bản thì hệ thống phản ứng thế nào ?
: kịch bản cơ sở
 - Người dùng làm khác kịch bản thì hệ thống phản ứng thế nào ?
: kịch bản thay thế
 - ...
- Use case được mô tả bởi văn bản, hoặc bằng bảng

3.1. Khái niệm Use case

- Use case mô tả một chuỗi các hành động của người dùng cùng với chuỗi các đáp ứng/phản hồi của hệ thống để thực hiện 1 việc, 1 chức năng nào đó của hệ thống.
- Nội dung use case giống như kịch bản/đoạn hội thoại mô tả cách sử dụng hệ thống: luôn có cấu trúc “người dùng làm gì – hệ thống phản ứng như thế nào”.
- Cần mô tả kỹ về cách người dùng sử dụng hệ thống:
 - Người dùng làm theo kịch bản thì hệ thống phản ứng thế nào ?
: kịch bản cơ sở
 - Người dùng làm khác kịch bản thì hệ thống phản ứng thế nào ?
: kịch bản thay thế
 - ...
- Use case được mô tả bởi văn bản, hoặc bằng bảng

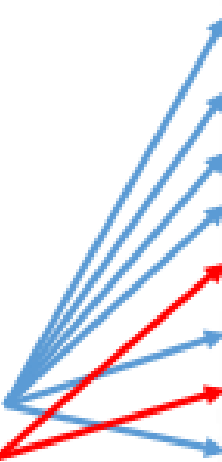
3.2. Mô tả Use case

- Tên use case: **“Buy a book”**

“Foster wants to buy a book. He goes to the Amazon website and browses to find his book. He adds it to his shopping cart. He then goes to the check-out page where he provides his billing information and confirms his purchase.”

Từng hành động trong use case:

- + Ai, làm gì, lên đối tượng nào
- + Hệ thống, làm gì, lên đối tượng nào



Actor	Action	Object
Foster	browses	Amazon website
Foster	selects	book
Foster	adds	book, shopping cart
Foster	checks out	
Amazon	requests	billing information
Foster	provides	name, address, credit
Amazon	verifies	credit information
Foster	confirms	purchase

3.3. Biểu đồ Use case

- Mô tả tập hợp các use case của hệ thống
 - Mỗi use case mô tả một yêu cầu chức năng của hệ thống
- Các thành phần chính:
 - Tác nhân : Thường là người dùng, hệ thống ngoài, thiết bị ngoài, có tương tác với hệ thống.
 - Các Use case
 - Đường bao hệ thống



3.3. Biểu đồ Use case

- Liên kết trong biểu đồ use case

- Liên kết không có nhãn: liên kết tác nhân tham gia vào use case



- Liên kết có nhãn <<extend>>: Use case B có thể được sử dụng trong use case A.



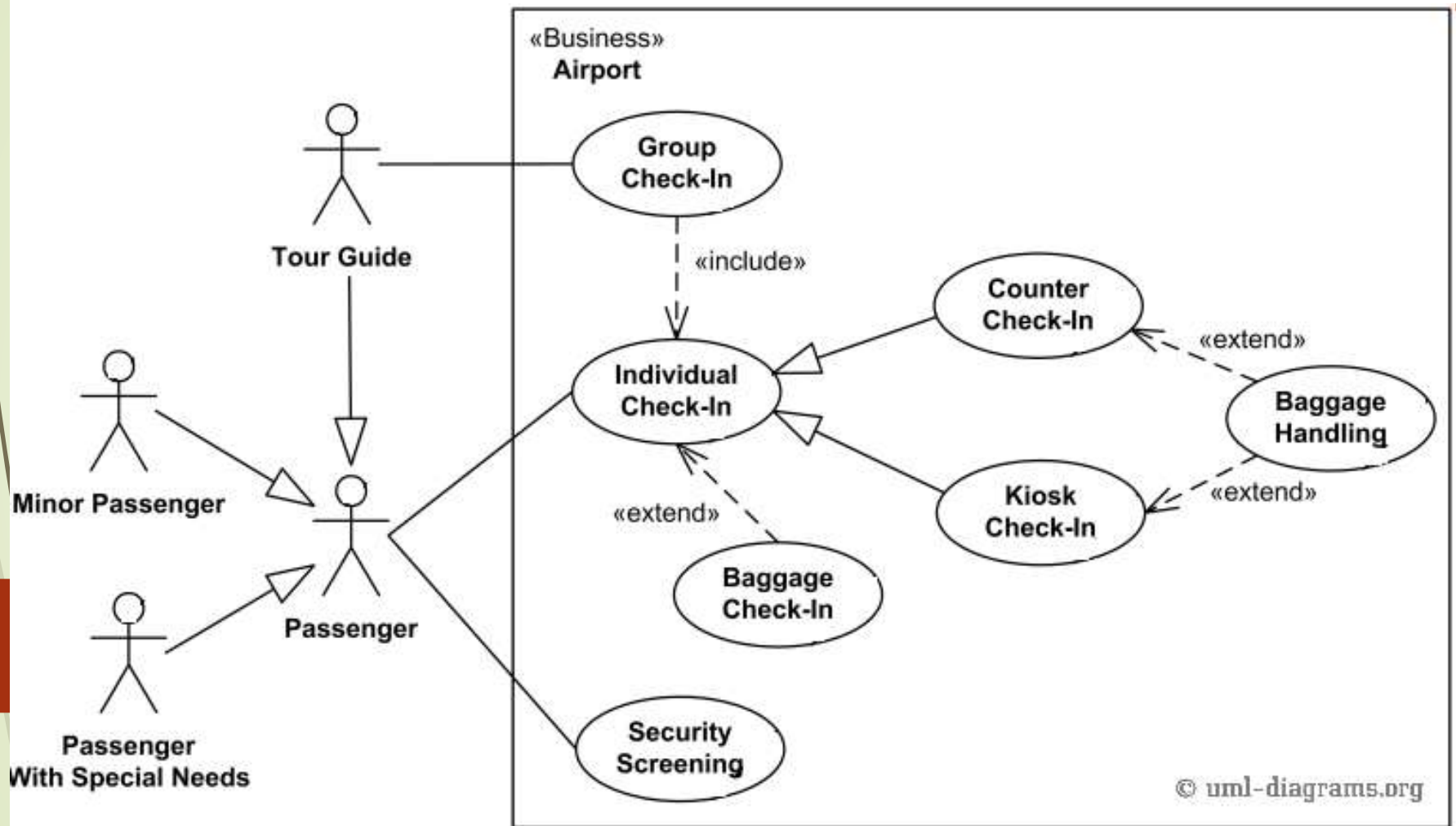
- Liên kết có nhãn <<include>>: Use case B luôn được sử dụng trong use case A.



- Liên kết tổng quát hóa: mô tả quan hệ khái quát hóa của tác nhân hoặc use case, A là khái quát hóa của B.



3.3. Biểu đồ Use case



Q&A: [Xem các ví dụ về Use case Diagram](#)

INCLUDE vs EXTEND

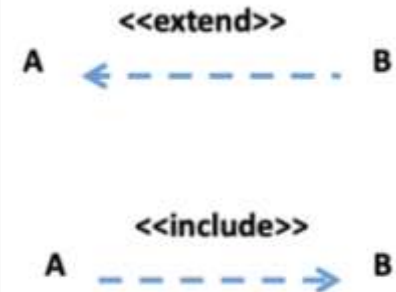
«include» — luôn luôn gọi

- Use case A luôn gọi use case B, không có ngoại lệ
- Mũi tên nét đứt từ A → B, nhãn «include»
- Ví dụ: "Checkout" luôn gọi "Payment" → Checkout --«include»→ Payment

«extend» — có điều kiện mới gọi

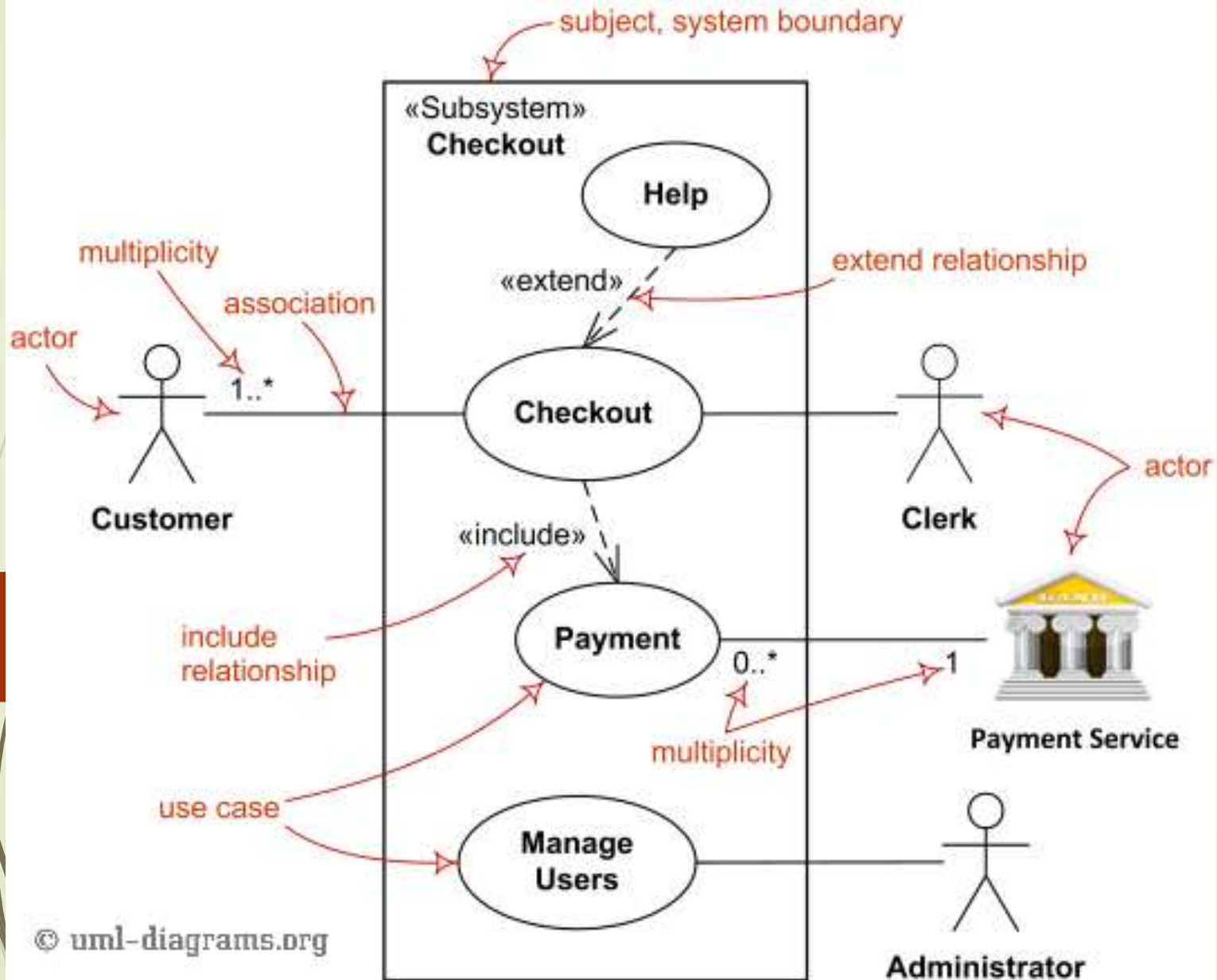
- Use case B chỉ được kích hoạt khi có điều kiện đặc biệt
- Mũi tên nét đứt từ B → A (ngược chiều!), nhãn «extend»
- Ví dụ: "Help" chỉ xuất hiện khi user bấm nút Trợ giúp → Help --«extend»→ Checkout

Tiêu chí	«include»	«extend»
Khi nào xảy ra?	Luôn luôn (bắt buộc)	Khi có điều kiện (tùy chọn)
Hướng mũi tên	A → B (A gọi B)	B → A (B mở rộng A)
Câu hỏi kiểm tra	"B có luôn chạy không?" → có: include	"B có thể không chạy không?" → có: extend
Ví dụ thực tế	Đăng nhập → Xác thực mật khẩu	Đặt hàng → Nhập mã giảm giá



🔑 Mẹo nhớ: include = included always (bắt buộc) | extend = extra optional (không bắt buộc)

3.3. Biểu đồ Use case



3.4. Cách xây dựng biểu đồ Use case

- Người phát triển phần mềm thống nhất với khách hàng về bản mô tả hệ thống chi tiết, các yêu cầu chức năng, phi chức năng, v.v.
- Xác định các use case từ bản mô tả hệ thống:
 - Mỗi use case mô tả một yêu cầu chức năng của hệ thống, chưa quan tâm tới việc chức năng đó được thực thi như thế nào.
 - Tên use case là một Cụm động từ, ví dụ “Mua hàng”
- Xác định các tác nhân bên ngoài có ảnh hưởng đến hệ thống
 - Tác nhân là nhóm người (hoặc hệ thống) có 1 vai trò rõ ràng. Tên tác nhân là Danh từ.
 - Ví dụ “Người mua”, “Quản trị viên” chứ không phải tên riêng của người dùng “Bob”, “John” ...
- Hoàn thiện các liên kết, đường bao hệ thống.

Bài tập

Cho:

- Các tác nhân: Người mua, Hệ thống E-mail, Hệ thống cho vay và Hệ thống báo cáo tín dụng
- Các use case: Tìm người môi giới, Quản lý hồ sơ cá nhân, Tìm kiếm nhà và Yêu cầu vay
- Các mối liên kết:
 - Từ người mua tới Tìm người môi giới
 - Từ người mua tới Quản lý hồ sơ cá nhân
 - Từ người mua tới Tìm kiếm nhà
 - Từ người mua tới Yêu cầu vay
 - Quản lý hồ sơ cá nhân tới Hệ thống e-mail
 - Tìm kiếm nhà tới Hệ thống e-mail
 - Yêu cầu vay tới Hệ thống e-mail, Hệ thống cho vay
 - Yêu cầu vay tới Hệ thống báo cáo tín dụng

Hãy vẽ biểu đồ use case tương ứng.

Bài tập

Chú ý: Hiện có rất nhiều phần mềm (Astar UML, StarUML), cũng như trang web online (Lucid chart, Smart Draw) cho phép vẽ các Class diagram, Use case diagram. Sinh viên có thể thử các nền tảng trên.

Phiên bản non-commercial for community dưới đây cũng có thể sử dụng.

<https://www.visual-paradigm.com/download/community.jsp>

4



Biểu đồ lớp

Class diagram

<https://www.smartdraw.com/class-diagram/>

https://www.youtube.com/watch?time_continue=1&v=Wl0oyCeon2A

4.1. Khái niệm lớp

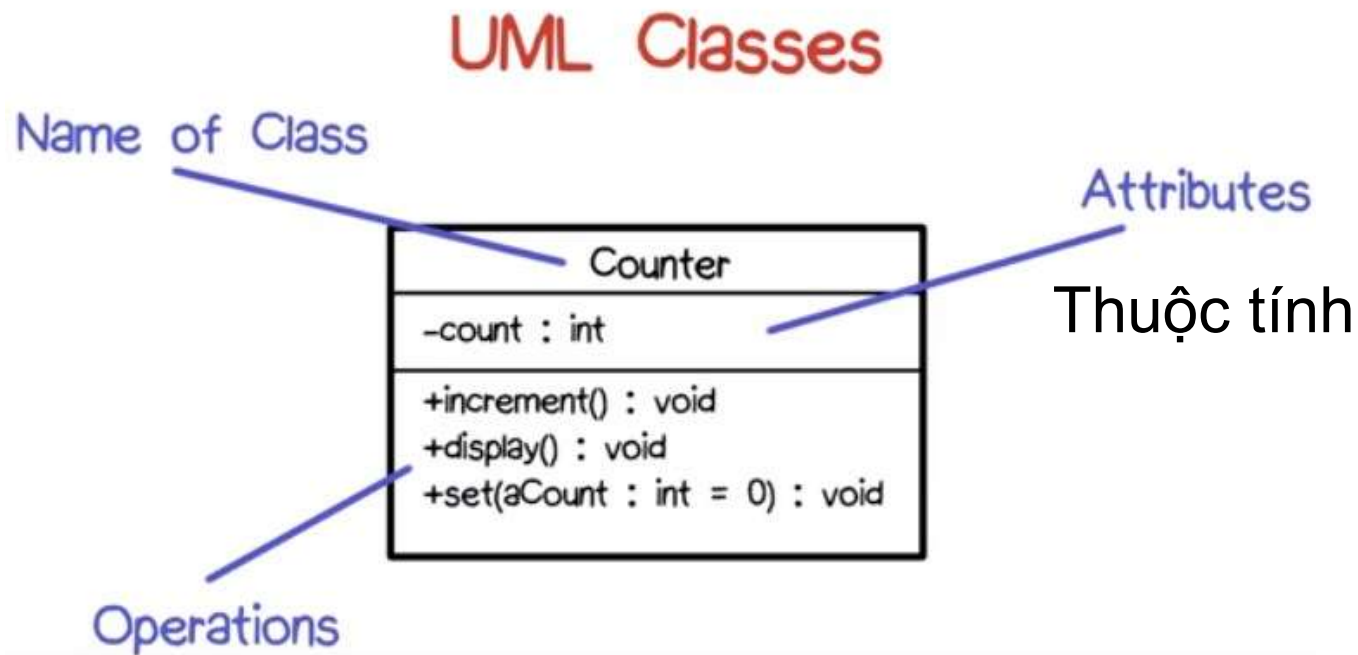
- Lớp là khái niệm mô tả 1 tập hợp các đối tượng mang cùng một ý nghĩa (bao gồm cả thuộc tính, hành vi/tác vụ, quan hệ).
- Mỗi một đối tượng sẽ là một thực thể / cá thể rõ ràng của lớp.
 - Mỗi sinh viên là một đối tượng với
 - các thuộc tính: tên, tuổi, khoa, lớp, khoá,...
 - các tác vụ: học bài, lên lớp, làm bài kiểm tra,...

Mô hình hóa → Lớp Student
 - Mỗi chiếc điện thoại là một đối tượng với
 - các thuộc tính: số SIM, model, kích thước,...
 - các tác vụ: gọi số, nhắn tin, nghe cuộc gọi tới, từ chối cuộc gọi,...

Mô hình hóa → Lớp SmartPhone

4.2. Biểu đồ lớp

- Dạng biểu đồ phổ biến nhất, mô tả cấu trúc tĩnh của hệ thống
- Biểu diễn các lớp và mối quan hệ giữa các lớp

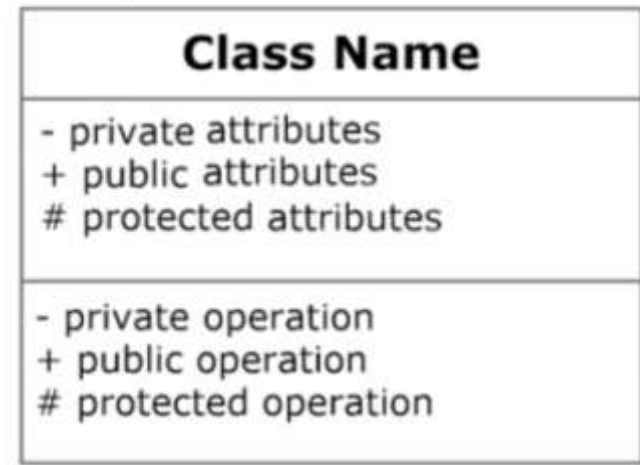
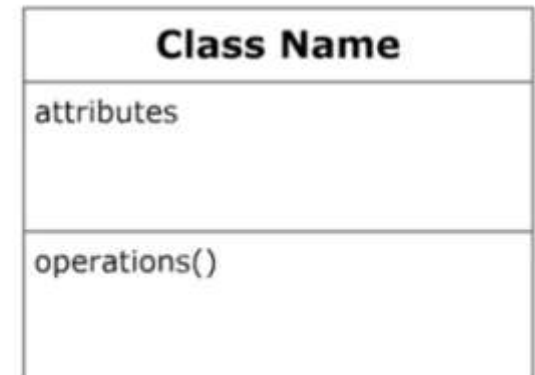


Tác vụ/ Phương thức

4.2. Biểu đồ lớp

Các ký hiệu

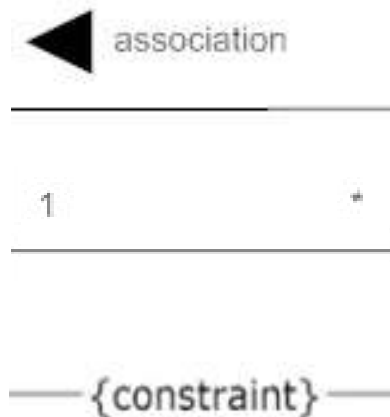
- Lớp: tên lớp, thuộc tính, tác vụ/phương thức
- Phạm vi truy cập: ai được truy cập các thông tin trong lớp
 - -: **private**: không được truy cập từ ngoài lớp
 - +: **public**: truy cập được từ bất kỳ lớp / phương thức ngoài lớp
 - #: **protected**: truy cập được từ các lớp thừa kế



4.2. Biểu đồ lớp

Quan hệ giữa các lớp

- Liên kết (Association): có kết nối, có sử dụng
 - Bội số quan hệ (Multiplicity)
 - Có hướng (navigability): chỉ rõ hướng liên kết
 - Ràng buộc (constraint): có điều kiện về liên kết

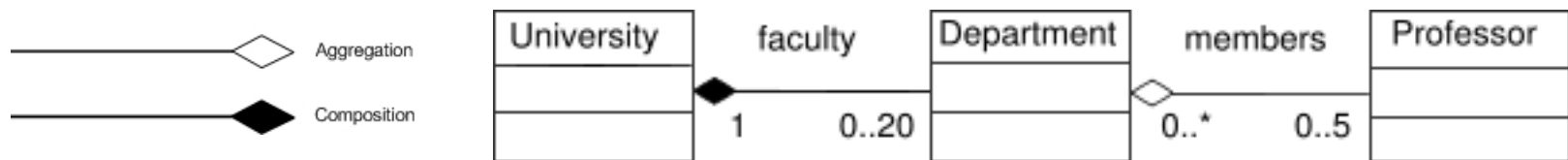


Indicator		Meaning
0..1		Zero or one
1		One only
0..*		0 or more
1..*	*	1 or more
n		Only n (where n > 1)
0..n		Zero to n (where n > 1)
1..n		One to n (where n > 1)

4.2. Biểu đồ lớp

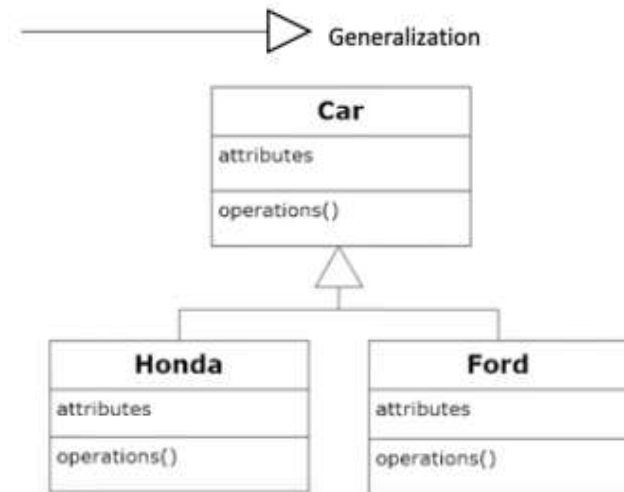
Quan hệ giữa các lớp

- Kết tập (Aggregation): mô tả quan hệ toàn thể - bộ phận hay còn gọi là quan hệ “is a part of”.
- Cấu thành (Composition): là một dạng kết tập, bộ phận không thể tồn tại nếu toàn thể bị hủy bỏ.



- Tổng quát hóa (Generalization): khi một lớp là một dạng cụ thể của một lớp khác. Hay còn gọi là quan hệ “is a kind of”

- Lớp con thừa hưởng các thuộc tính/phương thức public và protected của lớp mẹ



4.3. Cách phát hiện lớp

- Phân tích bản mô tả yêu cầu về hệ thống, phát hiện ra tập các đối tượng có thể có trong hệ thống (dựa trên các Danh từ, Ngữ danh từ).
- Phân tích bản mô tả use case của hệ thống, bổ sung các đối tượng phát sinh.
- Thống nhất các đối tượng trùng nhau
 - Ví dụ “shopping cart”, “shopping basket”, “shopping trolley” thành “shopping cart”
- Loại bỏ các đối tượng vô nghĩa, hoặc không đủ lớn thành một lớp
 - Ví dụ “Internet”, “Password”, “Title”, ...
- Giao diện giữa hệ thống và bên ngoài có thể coi là các đối tượng biên
 - Ví dụ: form đăng nhập, form đăng ký tín chỉ, v.v.
- Có thể hình thành các đối tượng điều khiển nếu cần
 - Ví dụ: đối tượng điều khiển đăng nhập `RegistrationController` v.v.
- Mô hình hóa các đối tượng thành các lớp, biểu diễn quan hệ giữa các lớp.

Ví dụ: Hệ thống đăng ký khóa học

• Các lớp cơ bản

CloseRegistrationForm
+ open() + close registration()

Student
+ get tuition() + add schedule() + get schedule() + delete schedule() + has pre-requisites()

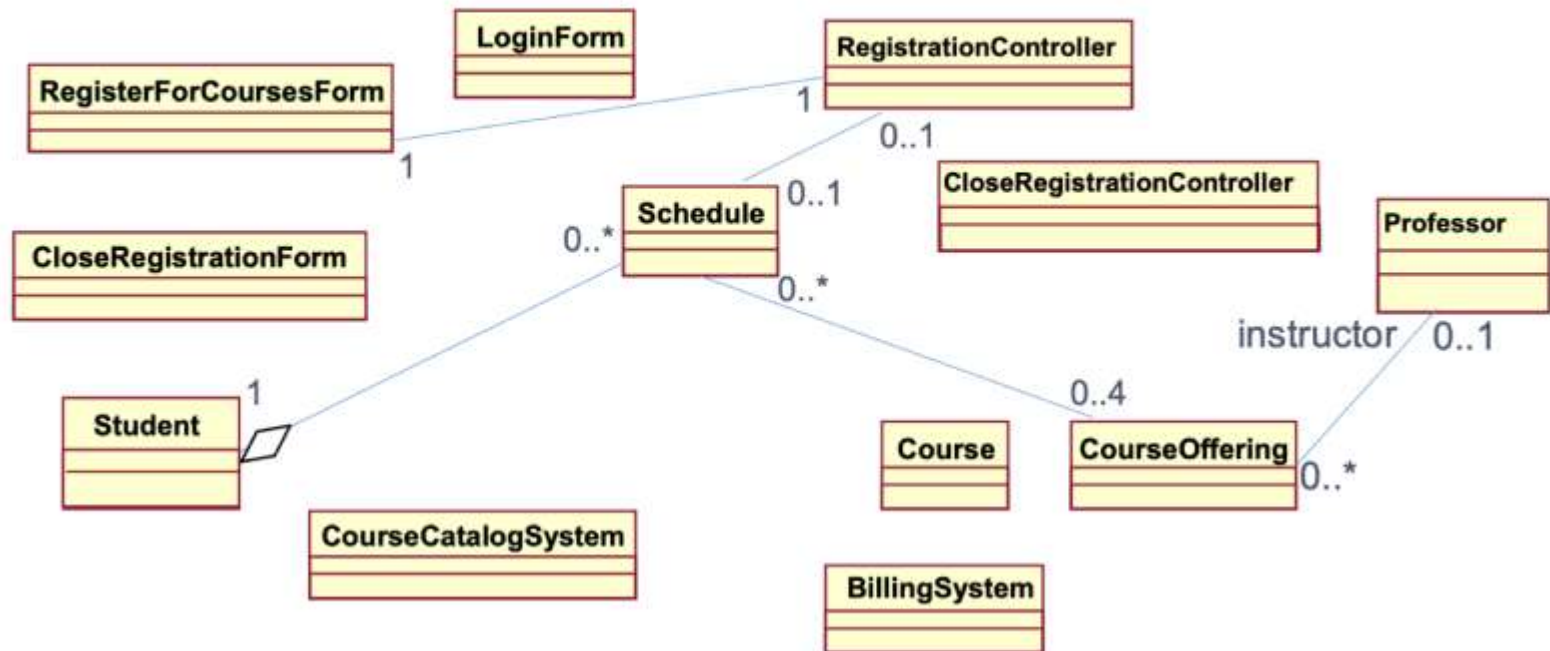
Schedule
- semester
+ commit() + select alternate() + remove offering() + level() + cancel() + get cost() + delete() + submit() + save() + any conflicts?() + create with offerings() + update with new selections()

CloseRegistrationController
+ is registration open?() + close registration()

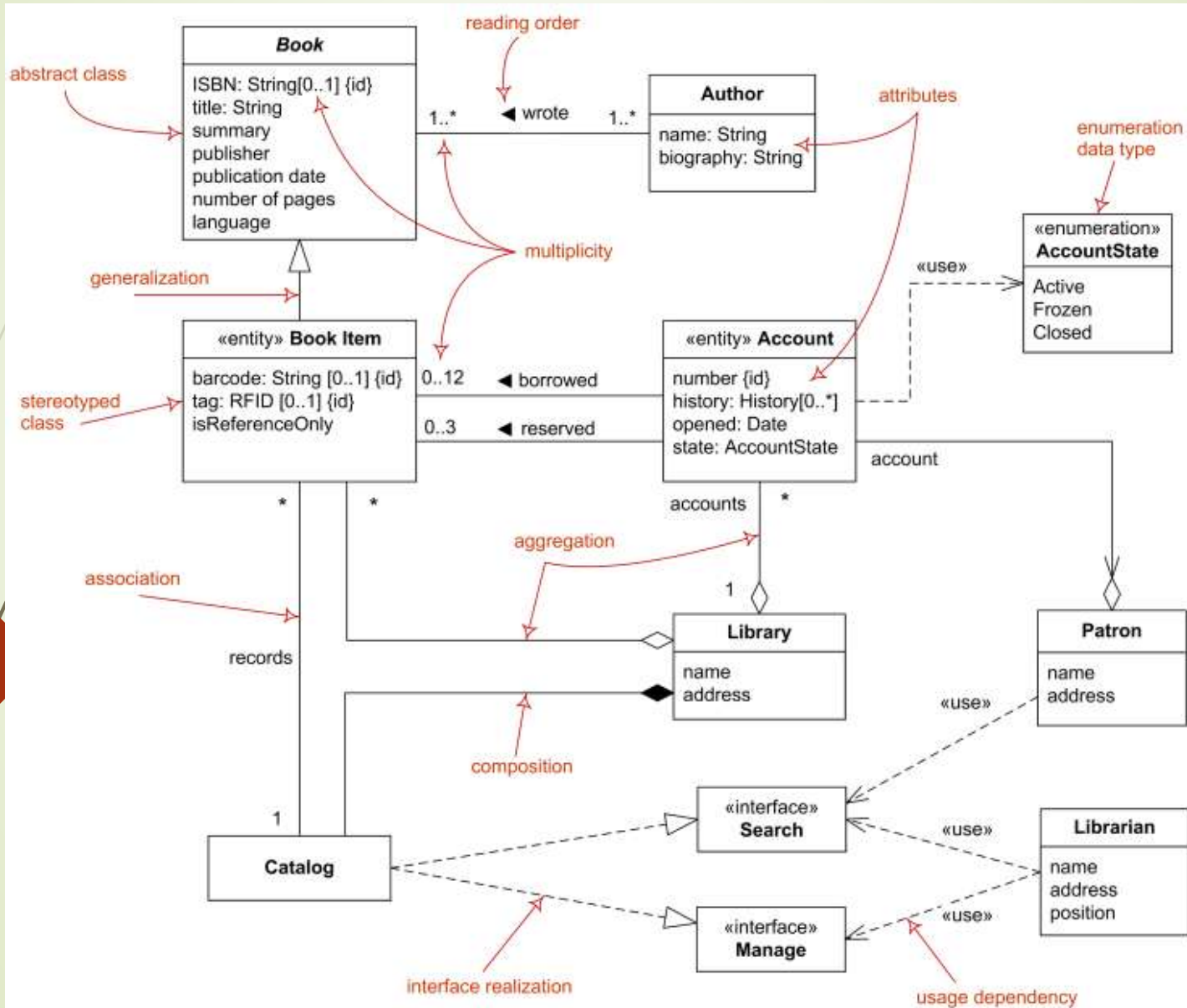
Professor
- name - employeeID : UniqueId - hireDate - status - discipline - maxLoad
+ submitFinalGrade() + acceptCourseOffering() + setMaxLoad() + takeSabbatical() + teachClass()

Ví dụ: Hệ thống đăng ký khóa học

- Biểu đồ lớp sơ lược



Ví dụ: Hệ thống quản lý sách

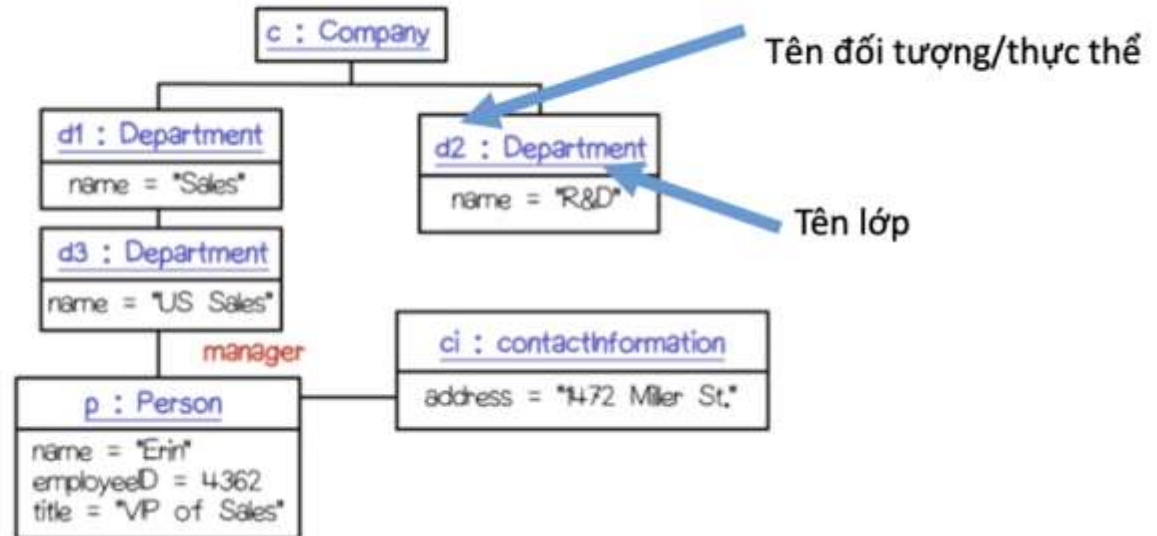


4.4. Biểu đồ đối tượng

- Giống biểu đồ lớp, thay vì biểu diễn lớp, biểu đồ đối tượng biểu diễn các thực thể thật của lớp và quan hệ giữa chúng

Object Diagram

Conveys **objects** and **links** instead of **classes** and **relationships**



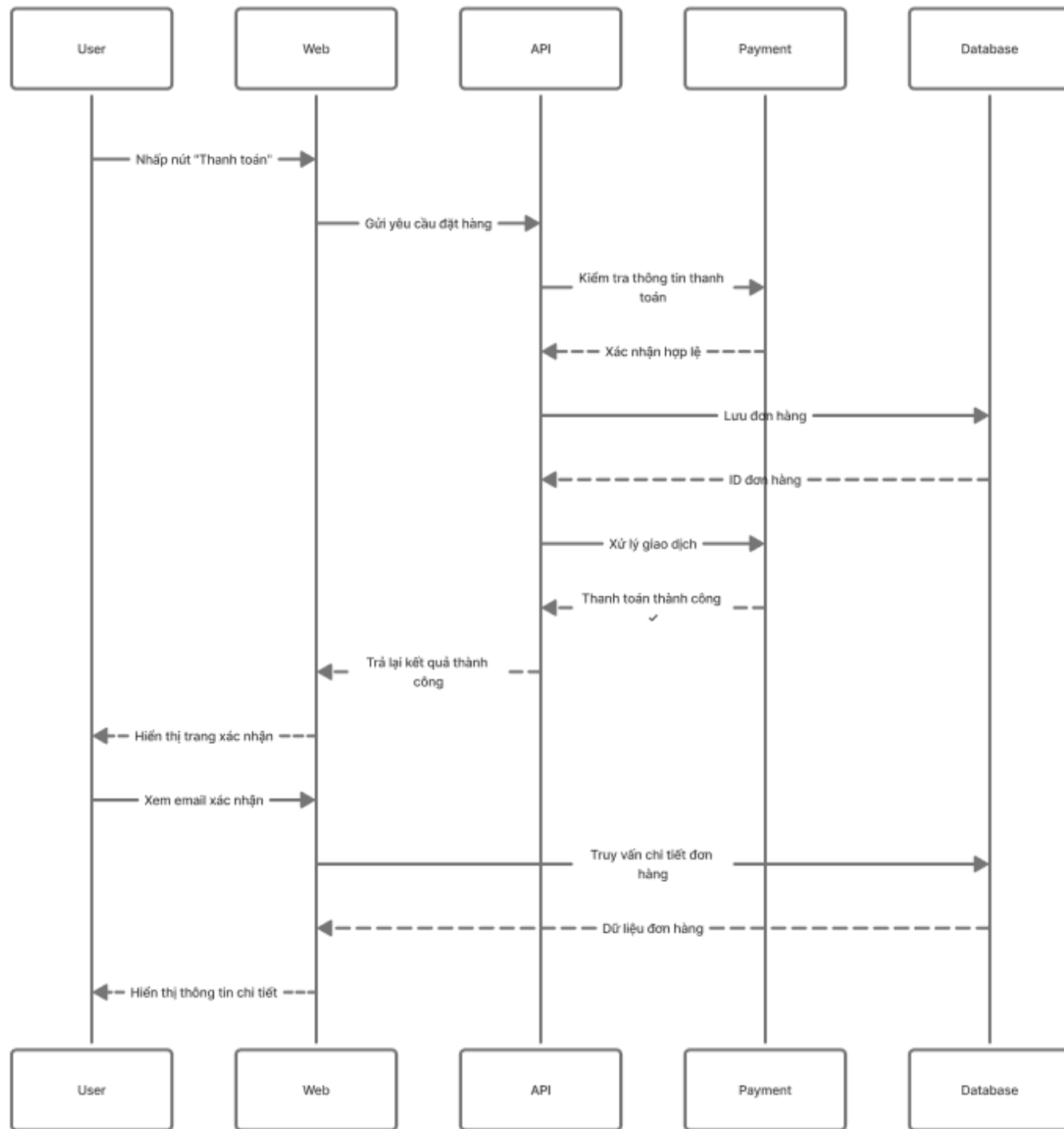
5

Biểu đồ tuần tự

Sequence diagram

<https://www.smartdraw.com/sequence-diagram/>

<https://www.youtube.com/watch?v=pCK6prSq8aw>



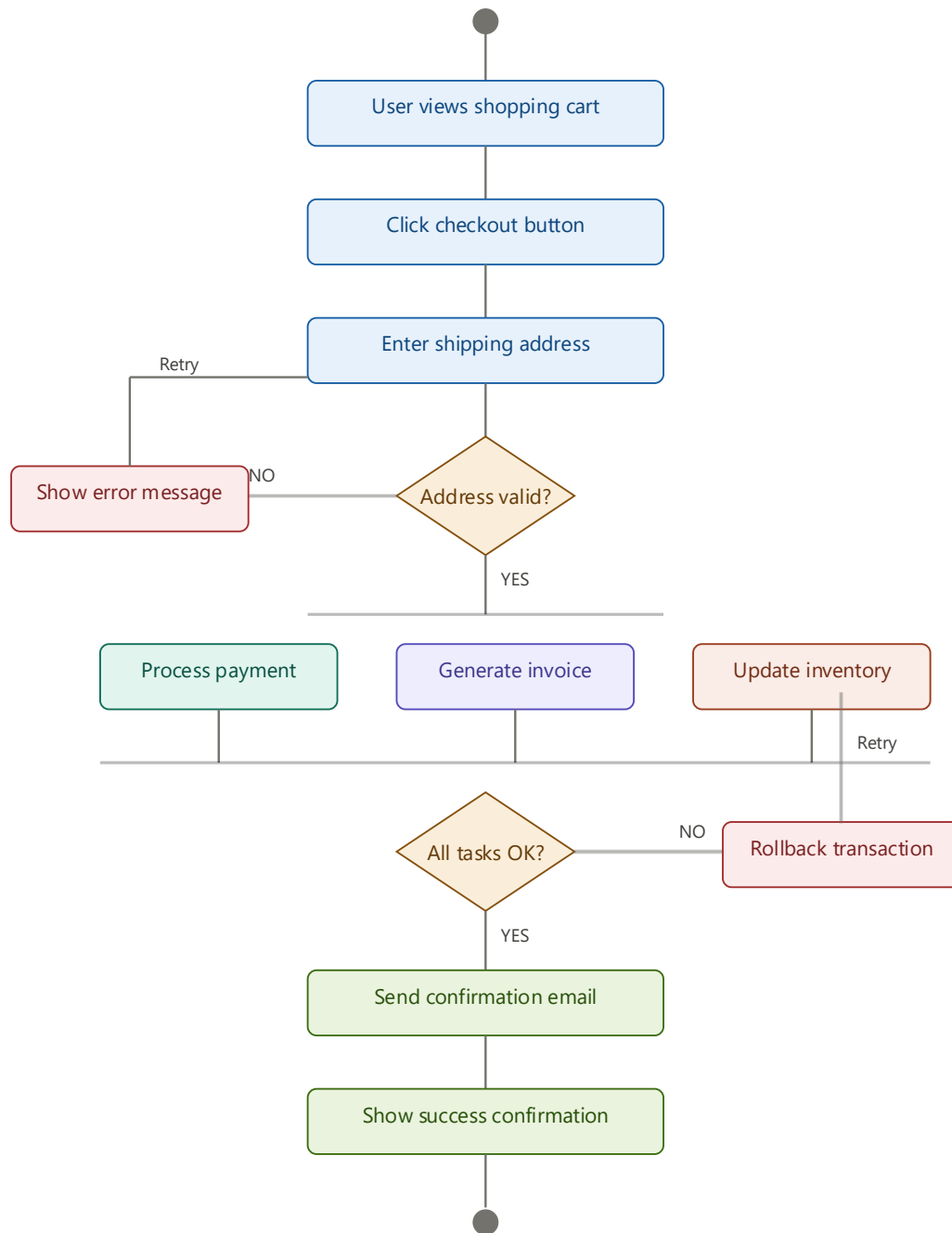
6

Biểu đồ hoạt động

Activity diagram

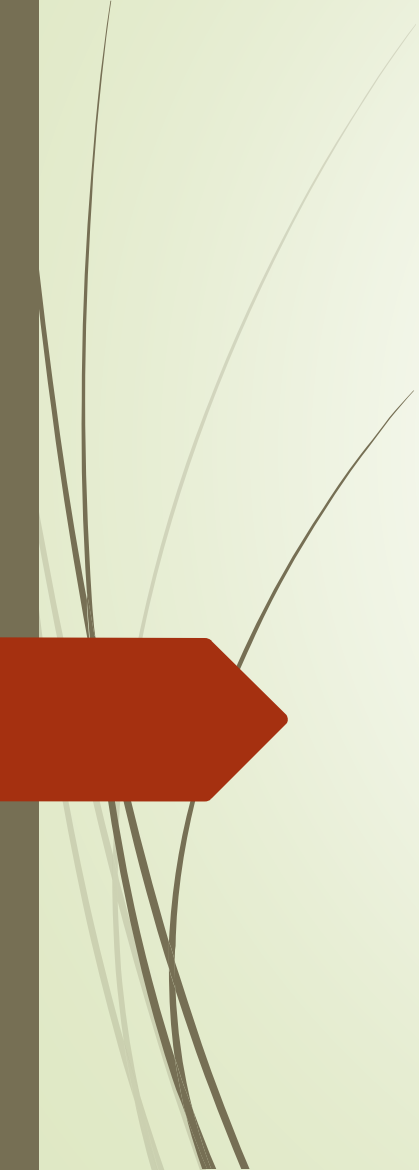
<https://www.smartdraw.com/activity-diagram/>

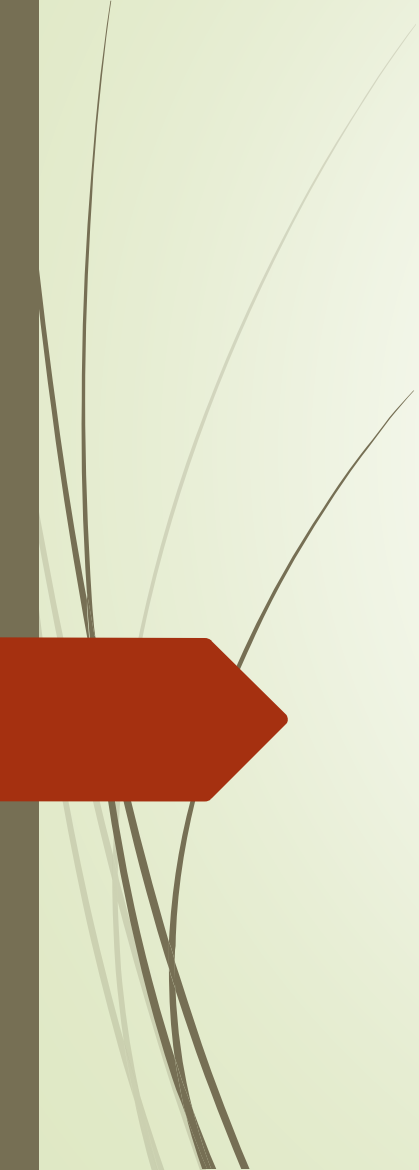
https://www.youtube.com/watch?v=Wf_xlagfHmg



Blue = user action | Amber = decision | Green = success | Red = error

Bài tập về nhà



A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

Thank you!

Any questions?